

Contents

Trabalho Prático: Pipeline de Eventos em Tempo Real com Go e Kafka	1
1. Contexto e Motivação	1
2. Objetivos de Aprendizagem	1
3. Descrição do Sistema	1
4. Requisitos OBRIGATÓRIOS	3
5. Requisitos OPCIONAIS (bônus)	4
6. Stack Tecnológica Sugerida	5
7. Estrutura de Pastas Esperada	5
8. Cronograma Sugerido	7
9. Critérios de Avaliação	7
10. Entregáveis	7
11. Dicas de Implementação	7
12. Referências	8
13. FAQ	8

Trabalho Prático: Pipeline de Eventos em Tempo Real com Go e Kafka

Disciplina: Engenharia de Software / Sistemas Distribuídos **Linguagem:** Go 1.22+ **Modalidade:** Projeto individual de portfólio **Duração estimada:** 4 semanas (40-60h de trabalho) **Nível:** Pleno / preparação para Sênior

1. Contexto e Motivação

Empresas que operam em escala (iFood, Mercado Livre, Nubank, Stone, Globo, etc.) processam **milhões de eventos por segundo**: localização de entregadores, transações financeiras, plays de vídeo, atualizações de pedidos. Esses eventos não podem ser perdidos, precisam ser processados de forma confiável, e o sistema precisa continuar funcionando mesmo quando partes dele falham.

Para resolver esse tipo de problema, o padrão de mercado é **arquitetura orientada a eventos (Event-Driven Architecture)** com **mensageria via Kafka**, microserviços desacoplados e observabilidade completa.

Este projeto simula um sistema de **rastreamento de pedidos em tempo real**, similar ao que o iFood usa pra acompanhar entregadores. O aluno deverá projetar e implementar uma pipeline distribuída em Go que demonstre domínio dos conceitos fundamentais usados em produção em empresas de tecnologia.

2. Objetivos de Aprendizagem

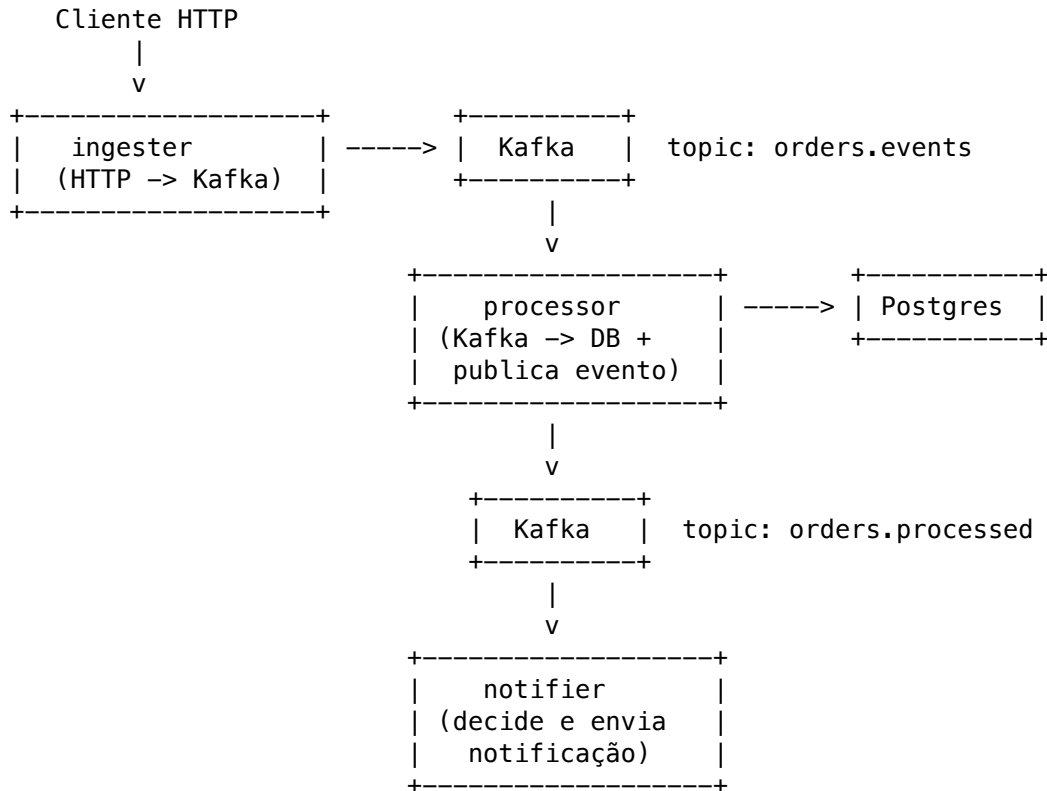
Ao concluir o projeto, o aluno deverá ser capaz de:

- Projetar e implementar microserviços em Go aplicando Clean Architecture
- Modelar comunicação assíncrona entre serviços usando Apache Kafka
- Implementar padrões de resiliência (retry, DLQ, idempotência, graceful shutdown)
- Instrumentar aplicações com observabilidade (logs estruturados, métricas, tracing distribuído)
- Escrever testes de integração com infraestrutura real (testcontainers)
- Containerizar e orquestrar uma stack completa com Docker Compose
- Configurar pipeline de CI/CD com GitHub Actions
- Documentar decisões técnicas de forma profissional

3. Descrição do Sistema

O sistema é composto por **3 microserviços** que se comunicam via Kafka:

3.1 Diagrama de Arquitetura



3.2 Serviços

3.2.1 ingestor — Serviço de Ingestão - Expõe API REST POST /events - Valida payload JSON - Publica evento no tópico Kafka orders.events - Responde HTTP 202 Accepted (fire-and-forget)

3.2.2 processor — Serviço de Processamento - Consome do tópico orders.events - Aplica regras de negócio (ex: atualiza status do pedido, calcula ETA) - Persiste no PostgreSQL - Publica novo evento no tópico orders.processed - Em caso de falha, encaminha para Dead Letter Queue (orders.events.dlq)

3.2.3 notifier — Serviço de Notificações - Consome do tópico orders.processed - Decide se deve notificar o cliente (regra: apenas em mudança de status) - Simula envio de notificação (log + persistência em tabela notifications)

3.3 Modelo de Evento (exemplo)

```
{
  "event_id": "uuid-v4",
  "order_id": "ORD-12345",
  "type": "location_update",
  "timestamp": "2026-06-02T14:30:00Z",
  "payload": {
    "lat": -23.5505,
    "lng": -46.6333,
    "status": "in_transit"
  }
}
```

4. Requisitos OBRIGATÓRIOS

Os requisitos abaixo são **obrigatórios** e compõem a nota base do trabalho.

4.1 Funcionalidades

- ☐ **RF-01** — O ingestor deve aceitar requisições POST em /events com payload JSON
- ☐ **RF-02** — O ingestor deve validar o payload (campos obrigatórios, tipos, formatos)
- ☐ **RF-03** — O ingestor deve publicar o evento no Kafka e responder 202 em < 100ms
- ☐ **RF-04** — O processor deve consumir eventos do Kafka e persistir no PostgreSQL
- ☐ **RF-05** — O processor deve publicar evento processado em orders.processed
- ☐ **RF-06** — O notifier deve consumir orders.processed e registrar notificações
- ☐ **RF-07** — Endpoint GET /healthz em todos os serviços (liveness probe)
- ☐ **RF-08** — Endpoint GET /readyz em todos os serviços (readiness probe)

4.2 Resiliência e Confiabilidade

- ☐ **RNF-01 — Idempotência:** o processor deve garantir que o mesmo event_id não seja processado duas vezes (mesmo que o Kafka entregue duplicado)
- ☐ **RNF-02 — Retry com backoff exponencial:** em caso de falha transitória (banco fora, timeout), o processor deve tentar novamente com intervalos crescentes (1s, 2s, 4s, 8s) por no mínimo 3 tentativas
- ☐ **RNF-03 — Dead Letter Queue:** após esgotar retries, o evento deve ir para orders.events.dlq com metadados do erro
- ☐ **RNF-04 — Graceful shutdown:** ao receber SIGTERM/SIGINT, os serviços devem terminar processamento em andamento, fazer commit de offsets, fechar conexões e sair com código 0 em até 30 segundos
- ☐ **RNF-05 — Context propagation:** usar context.Context em todas as operações de I/O, com timeouts apropriados

4.3 Persistência

- ☐ **RNF-06** — PostgreSQL como banco principal
- ☐ **RNF-07** — Migrations versionadas (golang-migrate ou equivalente)
- ☐ **RNF-08** — Pelo menos 3 tabelas: orders, order_events, notifications
- ☐ **RNF-09** — Uso de pgx/v5 como driver (não usar database/sql puro)
- ☐ **RNF-10** — Connection pool configurado adequadamente

4.4 Observabilidade

- ☐ **RNF-11 — Logs estruturados** em JSON usando log/slog (stdlib do Go)
- ☐ **RNF-12** — Logs devem incluir: timestamp, level, service, event_id, trace_id
- ☐ **RNF-13 — Métricas Prometheus** expostas em /metrics:
 - events_received_total (counter)
 - events_processed_duration_seconds (histogram)
 - events_failed_total (counter, label por tipo de erro)
- ☐ **RNF-14 — Tracing distribuído** com OpenTelemetry, exportando para Jaeger
- ☐ **RNF-15** — trace_id deve atravessar os 3 serviços (do ingestor até o notifier)

4.5 Qualidade de Código

- ☐ **RNF-16** — Aplicar **Clean Architecture** (separação domain / application / infrastructure)
- ☐ **RNF-17** — Cobertura de testes >= 60% (medida com go test -cover)
- ☐ **RNF-18** — Testes unitários da camada de domínio e application
- ☐ **RNF-19** — Pelo menos 1 teste de integração usando testcontainers-go (Kafka + Postgres reais)
- ☐ **RNF-20** — Linter golangci-lint passando sem erros
- ☐ **RNF-21** — go vet e go fmt aplicados em todo o código

4.6 Infraestrutura e DevOps

- ☐ **RNF-22** — Dockerfile multi-stage para cada serviço (imagem final < 50MB)
- ☐ **RNF-23** — docker-compose.yml que sobe toda a stack: Kafka, Zookeeper (ou KRaft), Postgres, Prometheus, Grafana, Jaeger e os 3 serviços
- ☐ **RNF-24** — Configuração via **variáveis de ambiente** (12-factor app)
- ☐ **RNF-25** — Pipeline de **CI no GitHub Actions** rodando: build, lint, testes, cobertura

4.7 Documentação

- ☐ **RNF-26** — README.md com:
 - Descrição do projeto
 - Diagrama de arquitetura
 - Como rodar local (docker-compose up)
 - Exemplos de requisições (curl)
 - Stack tecnológica
 - Decisões técnicas (mini-ADRs)
 - ☐ **RNF-27** — Pelo menos **3 ADRs** (Architecture Decision Records) explicando decisões importantes
 - ☐ **RNF-28** — Documentação da API (OpenAPI/Swagger ou ao menos uma collection do Postman/Insomnia)
-

5. Requisitos OPCIONAIS (bônus)

Os requisitos abaixo são **opcionais** e compõem nota extra. Implementar 3-4 deles transforma o projeto em destaque para portfólio sênior.

5.1 Resiliência Avançada

- ☐ **OPC-01** — **Circuit Breaker** no acesso ao Postgres (lib sony/gobreaker)
- ☐ **OPC-02** — **Rate limiting** no ingestor usando Redis (token bucket)
- ☐ **OPC-03** — **Outbox Pattern**: garantir consistência entre escrita no banco e publicação no Kafka via tabela outbox
- ☐ **OPC-04** — **Saga Pattern** para um fluxo de cancelamento de pedido (compensação)

5.2 Observabilidade Avançada

- ☐ **OPC-05** — Dashboard **Grafana** versionado no repo (JSON commitado) com painéis de throughput, latência (p50/p95/p99), taxa de erro e lag do consumer
- ☐ **OPC-06** — **Alertas Prometheus** (alertmanager) para condições críticas (ex: lag > 1000, taxa de erro > 5%)
- ☐ **OPC-07** — **Profiling** habilitado via net/http/pprof em endpoint protegido

5.3 Performance e Escala

- ☐ **OPC-08** — **Benchmarks** (go test -bench) das operações críticas
- ☐ **OPC-09** — **Load test** com k6 ou Vegeta, com resultados documentados
- ☐ **OPC-10** — **Particionamento** correto no Kafka usando order_id como chave (garante ordem por pedido)
- ☐ **OPC-11** — **Batch processing** no consumer (consumir N eventos antes de commit)

5.4 Segurança

- ☐ **OPC-12** — **Autenticação JWT** no ingestor
- ☐ **OPC-13** — **TLS** entre serviços
- ☐ **OPC-14** — **Secrets** gerenciados via Docker secrets ou Vault (nunca em env vars hardcoded)
- ☐ **OPC-15** — **Validação de entrada** com go-playground/validator

5.5 Deploy e Cloud

- ☐ **OPC-16** — Deploy real de pelo menos 1 serviço em **AWS** (EC2 free tier, ECS Fargate ou Lambda)
- ☐ **OPC-17** — **Infraestrutura como código** com Terraform ou AWS CDK
- ☐ **OPC-18** — **Helm chart** ou manifestos Kubernetes para os serviços
- ☐ **OPC-19** — Pipeline de **CD** que faz deploy automático em ambiente de staging

5.6 Funcionalidades Extras

- ☐ **OPC-20** — Endpoint **gRPC** equivalente ao REST do ingester (mesma feature, dois protocolos)
- ☐ **OPC-21** — **WebSocket** para streaming de eventos em tempo real para o frontend
- ☐ **OPC-22** — **Frontend** simples (React ou Vue) consumindo o sistema
- ☐ **OPC-23** — **CLI administrativa** em Cobra para inspecionar eventos, reprocessar DLQ, etc.

5.7 Qualidade Avançada

- ☐ **OPC-24** — Cobertura de testes $\geq 80\%$
- ☐ **OPC-25** — **Mutation testing** com gremlins
- ☐ **OPC-26** — **Property-based testing** com gopter em pelo menos um componente
- ☐ **OPC-27** — **Chaos testing** simulando falhas (Postgres caindo, Kafka indisponível) e verificando recuperação

6. Stack Tecnológica Sugerida

Camada	Ferramenta	Obrigatório?
Linguagem	Go 1.22+	Sim
Mensageria	Apache Kafka	Sim
Cliente Kafka	segmentio/kafka-go	Sim
Banco de dados	PostgreSQL 16	Sim
Driver DB	jackc/pgx/v5	Sim
Migrations	golang-migrate/migrate	Sim
HTTP Router	go-chi/chi ou net/http	Sim
Tracing	OpenTelemetry SDK + Jaeger	Sim
Métricas	prometheus/client_golang	Sim
Logs	log/slog (stdlib)	Sim
Testes	stretchr/testify	Sim
Testes integração	testcontainers-go	Sim
Containers	Docker + Docker Compose	Sim
CI	GitHub Actions	Sim
Linter	golangci-lint	Sim
Cache / Rate Limit	Redis	Opc
Circuit Breaker	sony/gobreaker	Opc
Validator	go-playground/validator	Opc
Load test	k6 ou Vegeta	Opc
IaC	Terraform ou AWS CDK	Opc

7. Estrutura de Pastas Esperada

```
event-pipeline/  
├── cmd/  
│   ├── ingester/  
│   │   └── main.go
```

```

├── processor/
│   └── main.go
├── notifier/
│   └── main.go
├── internal/
│   ├── domain/                # entidades e regras de negócio puras
│   │   ├── event.go
│   │   └── order.go
│   ├── application/          # casos de uso (use cases)
│   │   ├── ingest_event.go
│   │   └── process_event.go
│   ├── infrastructure/
│   │   ├── kafka/
│   │   │   ├── producer.go
│   │   │   └── consumer.go
│   │   ├── postgres/
│   │   │   ├── order_repository.go
│   │   │   └── event_repository.go
│   │   └── http/
│   │       ├── handler.go
│   │       └── middleware.go
│   ├── observability/
│   │   ├── tracing.go
│   │   ├── metrics.go
│   │   └── logger.go
├── deployments/
│   ├── docker-compose.yml
│   ├── prometheus/
│   │   └── prometheus.yml
│   └── grafana/
│       ├── dashboards/
│       └── overview.json
├── migrations/
│   ├── 001_create_orders.up.sql
│   ├── 001_create_orders.down.sql
│   └── ...
├── docs/
│   ├── adr/
│   │   ├── 001-clean-architecture.md
│   │   ├── 002-idempotency-strategy.md
│   │   └── 003-retry-policy.md
│   └── architecture.png
├── scripts/
│   └── load-test.js
├── .github/
│   └── workflows/
│       └── ci.yml
├── Makefile
├── go.mod
├── go.sum
└── .golangci.yml

```

8. Cronograma Sugerido

Semana	Foco	Entregas
1	Fundação	docker-compose com infra; ingestor e processador básicos rodando
2	Robustez	Idempotência, retry, DLQ, graceful shutdown, notifier
3	Observabilidade	OpenTelemetry, Prometheus, Grafana, logs estruturados
4	Qualidade e finalização	Testes (unitários + integração), CI, README, ADRs, deploy opcional

9. Critérios de Avaliação

9.1 Distribuição de Pontos (base 10)

Critério	Peso
Funcionalidades obrigatórias (RF-01 a RF-08)	2.0
Resiliência (idempotência, retry, DLQ)	2.0
Observabilidade (logs, métricas, tracing)	1.5
Qualidade de código e testes	1.5
Arquitetura (Clean Architecture aplicada)	1.0
Infraestrutura (Docker, CI)	1.0
Documentação (README + ADRs)	1.0
Total base	10.0
Bônus por requisitos opcionais	até +3.0

9.2 Rubrica Resumida

- **Nota 10:** todos os obrigatórios + 4+ opcionais + deploy real funcionando
 - **Nota 8-9:** todos os obrigatórios bem implementados + 1-2 opcionais
 - **Nota 6-7:** maioria dos obrigatórios, falhas pontuais em resiliência ou testes
 - **Nota < 6:** falta funcionalidade core (ex: não tem idempotência, não tem testes)
-

10. Entregáveis

1. **Repositório Git público** (GitHub/GitLab) com todo o código
 2. **README** completo com instruções de setup
 3. **Vídeo de 5-10 minutos** demonstrando o sistema funcionando (opcional mas recomendado)
 4. **Diagrama de arquitetura** (formato Mermaid no README ou imagem)
 5. **ADRs** documentando decisões técnicas
 6. **Pipeline de CI passando** (badge verde no README)
-

11. Dicas de Implementação

11.1 Por onde começar

1. Sobe o docker-compose com Kafka, Postgres, Prometheus, Grafana e Jaeger **antes** de escrever qualquer código Go

2. Faz um “hello world” do Kafka: produzir e consumir uma mensagem simples
3. Só então começa a estruturar o projeto com Clean Architecture

11.2 Armadilhas comuns

- **Não usar context.Context desde o início** -> vira pesadelo refatorar depois
- **Esquecer do graceful shutdown** -> é o que diferencia código de produção de código de tutorial
- **Misturar camadas** -> manter o domínio puro (sem imports de Kafka/Postgres) é o que faz Clean Architecture funcionar
- **Não testar caminhos de erro** -> 80% dos bugs em produção estão nos caminhos de exceção
- **Logar dados sensíveis** -> revisar logs antes de commitar dashboards/exemplos

11.3 Como vender no portfólio

No README, abrir com uma seção tipo:

Este projeto demonstra construção de um sistema distribuído tolerante a falhas em Go, aplicando padrões usados em produção em empresas como iFood, Mercado Livre e Nubank: arquitetura orientada a eventos, idempotência, retry com backoff, dead letter queue, graceful shutdown, observabilidade end-to-end com OpenTelemetry e testes de integração com infraestrutura real.

Recrutador técnico lê isso e já marca entrevista.

12. Referências

- **Clean Architecture** — Robert C. Martin
 - **Designing Data-Intensive Applications** — Martin Kleppmann (capítulos 4, 11 e 12)
 - **Kafka: The Definitive Guide** — Neha Narkhede et al.
 - **Documentação OpenTelemetry Go**: <https://opentelemetry.io/docs/languages/go/>
 - **Effective Go**: https://go.dev/doc/effective_go
 - **Go Project Layout**: <https://github.com/golang-standards/project-layout>
 - **12-Factor App**: <https://12factor.net/>
-

13. FAQ

P: Posso usar uma alternativa ao Kafka (RabbitMQ, NATS)? R: Para o objetivo de portfólio, manter Kafka — é o que aparece em 90% das vagas em SP. RabbitMQ pode ser feito em um segundo projeto.

P: Posso usar gorm em vez de pgx? R: Não recomendado. pgx é o padrão da comunidade Go moderna e é o que você verá em entrevistas técnicas. ORMs pesados são vistos com ressalva.

P: Preciso fazer deploy real? R: Não é obrigatório, mas **transforma seu portfólio**. Hospedar pelo menos o ingerter numa EC2 free tier ou Fly.io permite que o recrutador clique e veja funcionando.

P: Posso usar uma IA pra escrever o código? R: Pode usar como ferramenta de produtividade, mas **você precisa entender cada linha**. Em entrevista, o recrutador vai perguntar “por que você escolheu X em vez de Y?”. Se você não souber explicar, perdeu o ponto.

Bom projeto e bons commits!